

Reporte de Teste de Intrusão

Laboratório: Guided Pentest: Web

Plataforma: Try Hack Me

Autor: Gabriel Alberto do Nascimento

Data: 12/08/2026

Versão: 1.0

Resumo Executivo

Foi realizado um teste de invasão (Pentest) do tipo Black Box na aplicação web RecruitX, plataforma destinada ao gerenciamento de vagas de emprego e candidaturas. O objetivo da avaliação foi identificar vulnerabilidades que pudessem comprometer a confidencialidade, integridade e disponibilidade das informações processadas pela aplicação, tendo como escopo a aplicação web e seu servidor de hospedagem.

Durante a avaliação foram identificadas vulnerabilidades que, quando exploradas em conjunto, permitiram o comprometimento da aplicação. Entre os principais achados destacam-se uma falha de controle de acesso (IDOR), exposição inadequada do mecanismo de redefinição de senha, falhas na validação de uploads de arquivos e exposição de endpoints da API sem controles adequados de autenticação e autorização.

A exploração encadeada dessas vulnerabilidades possibilitou a obtenção de acesso administrativo à aplicação e, posteriormente, a execução remota de comandos no servidor, demonstrando que um atacante poderia comprometer completamente o ambiente utilizando técnicas relativamente simples.

Caso essas vulnerabilidades permaneçam sem correção, a organização estará sujeita ao acesso não autorizado a informações sensíveis de candidatos e administradores, comprometimento do servidor de hospedagem, vazamento de dados pessoais, interrupção de serviços e possíveis impactos financeiros, operacionais e reputacionais, incluindo riscos relacionados ao cumprimento da Lei Geral de Proteção de Dados (LGPD).

Recomenda-se a implementação de controles adequados de autenticação e autorização em todas as funcionalidades da aplicação, a revisão completa do processo de redefinição de senhas, o fortalecimento das validações de upload de arquivos, a restrição do acesso aos endpoints da API e a realização de uma nova avaliação de segurança após a implementação das correções para validar a eficácia das medidas adotadas.

Descrição das Vulnerabilidades

IDOR em perfis de usuários e API

Classificação de Risco: Alto

Resumo: Foi identificada uma falha de controle de acesso do tipo **Insecure Direct Object Reference (IDOR)** que permite a um usuário autenticado acessar informações pertencentes a outros usuários por meio da manipulação do parâmetro `id` presente nas requisições HTTP. A vulnerabilidade também está presente na API da aplicação, permitindo a enumeração de usuários sem validação adequada de autorização.

Contexto: Uma vulnerabilidade do tipo **Insecure Direct Object Reference (IDOR)** ocorre quando uma aplicação utiliza identificadores fornecidos pelo usuário para acessar recursos internos sem realizar validações de autorização no lado do servidor. Como consequência, um usuário autenticado pode acessar informações pertencentes a terceiros simplesmente modificando esses identificadores e no caso da API até mesmo sem estar autenticado consegue acessar informações pertencentes a terceiros.

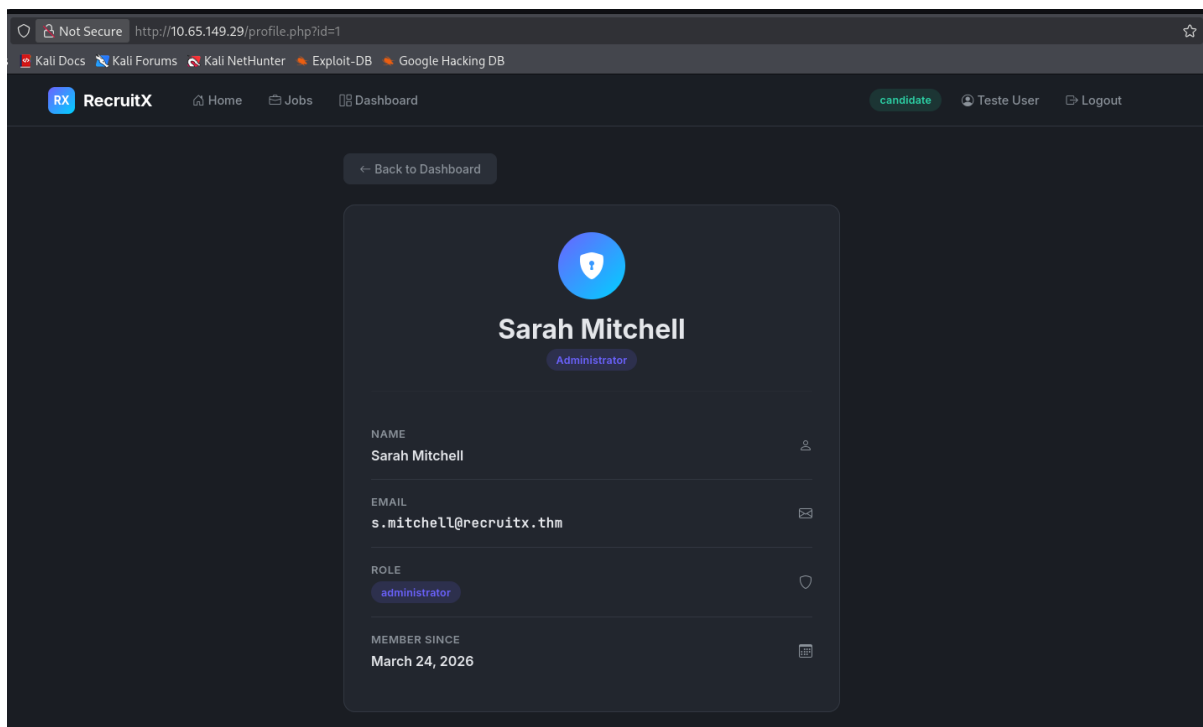
Detalhes técnicos e evidências:

Ao acessar o profile e notar a URL temos a seguinte forma:

<http://10.65.149.29/profile.php?id=7>

Ao tentar alterar o parâmetro ID o comportamento esperado de uma aplicação segura seria verificar se meu usuário tem permissão para ver aquele ID e caso tenha de mostrar e caso não tenha de dar um não permitido para o usuário.

Resultado ao utilizar <http://10.65.149.29/profile.php?id=1> :



Utilizando um cookie de sessão válido (PHPSESSID) com Curl podemos enumerar também os usuários tendo a possibilidade de montar um script para enumerar todos.

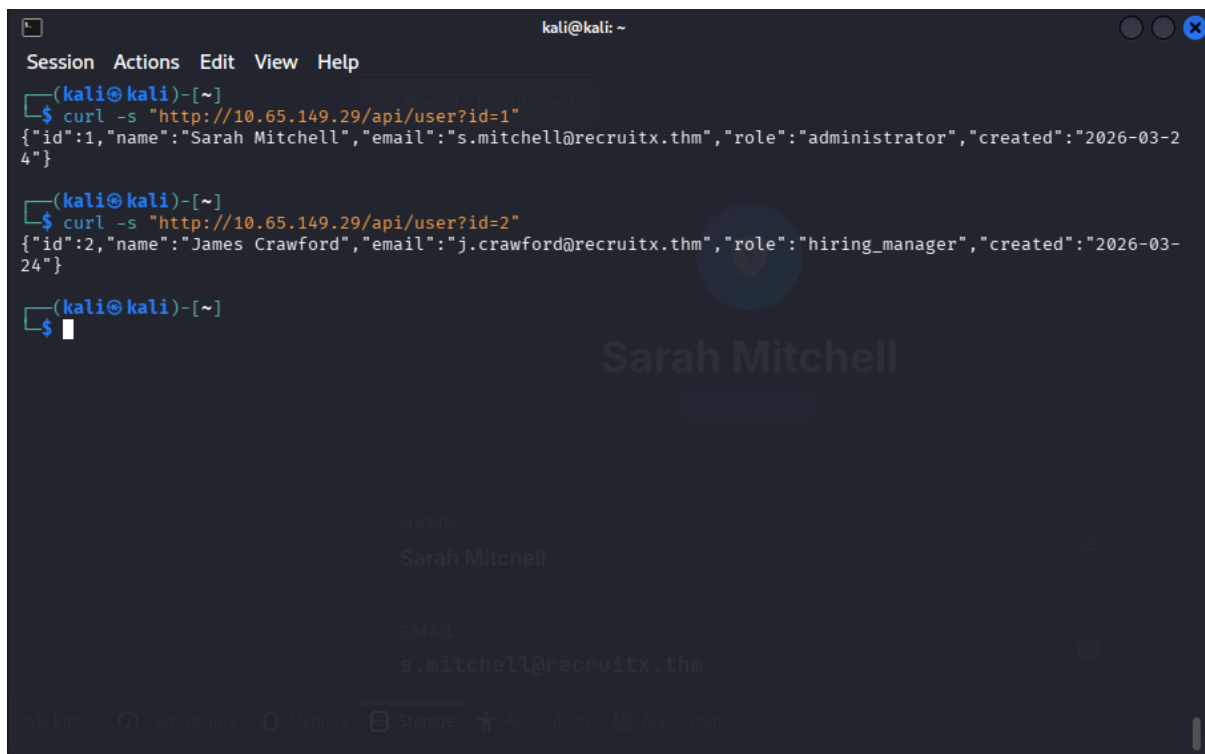
```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ curl -s -b "PHPSESSID=kldng4lqbe9hd72t6asdto95a9" "http://10.65.149.29/profile.php?id=1" | grep "fw-semib  
old"  
  
<div class="fw-semibold mt-1">Sarah Mitchell</div>  
<div class="fw-semibold mt-1 mono">s.mitchell@recruitx.thm</div>  
<div class="fw-semibold mt-1">March 24, 2026</div>  
  
(kali@kali)-[~]  
$
```

Essa vulnerabilidade também é replicada na rota API. As rotas foram descobertas da seguinte forma:

Comando: `curl http://10.65.136.150/api/`

Resultado: `{"endpoints":["Vapi\User","Vapi\jobs","Vapi\applications"]}`

Sendo assim acessando essa rota e validando os usuários conseguimos enumerar os usuários sem autenticação:



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ curl -s "http://10.65.149.29/api/user?id=1"  
{ "id":1,"name":"Sarah Mitchell","email":"s.mitchell@recruitx.thm","role":"administrator","created":"2026-03-24"}  
(kali@kali)-[~]  
$ curl -s "http://10.65.149.29/api/user?id=2"  
{ "id":2,"name":"James Crawford","email":"j.crawford@recruitx.thm","role":"hiring_manager","created":"2026-03-24"}  
(kali@kali)-[~]  
$
```

Impacto: A vulnerabilidade permite que usuários autenticados obtenham informações pertencentes a outros usuários, incluindo administradores. Essas informações podem ser utilizadas para reconhecimento do ambiente, engenharia social, ataques direcionados, comprometimento de contas e exploração de outras vulnerabilidades presentes na aplicação, aumentando significativamente o impacto da cadeia de ataque.

Recomendações de Remediação:

O servidor deve validar, para cada requisição, se o usuário autenticado possui autorização para acessar o recurso solicitado. A autorização não deve depender de verificações realizadas no cliente (frontend), mas exclusivamente de controles implementados no servidor. Requisições não autorizadas devem retornar HTTP 403 (Forbidden), impedindo o acesso ao recurso.

Exemplo de implementação em python flask:

```

1  from flask import abort
2  from flask_login import current_user
3
4  @app.route("/profile/<int:user_id>")
5  @login_required
6  def profile(user_id):
7      # Busca o perfil solicitado
8      profile = User.query.get_or_404(user_id)
9
10     # Verifica se o usuário autenticado possui autorização
11     if current_user.id != profile.id and not current_user.is_admin:
12         abort(403)
13
14     return render_template("profile.html", user=profile)

```

Referências:

https://owasp.org/www-community/attacks/insecure_direct_object_reference

Exposição de Token de Recuperação de Senha

Classificação de Risco: Crítico

Resumo: Foi identificada uma falha no mecanismo de recuperação de senha que expõe o token de redefinição diretamente na resposta da aplicação. Como consequência, um atacante que conheça o endereço de e-mail de um usuário válido pode utilizar esse token para redefinir sua senha e assumir o controle da conta.

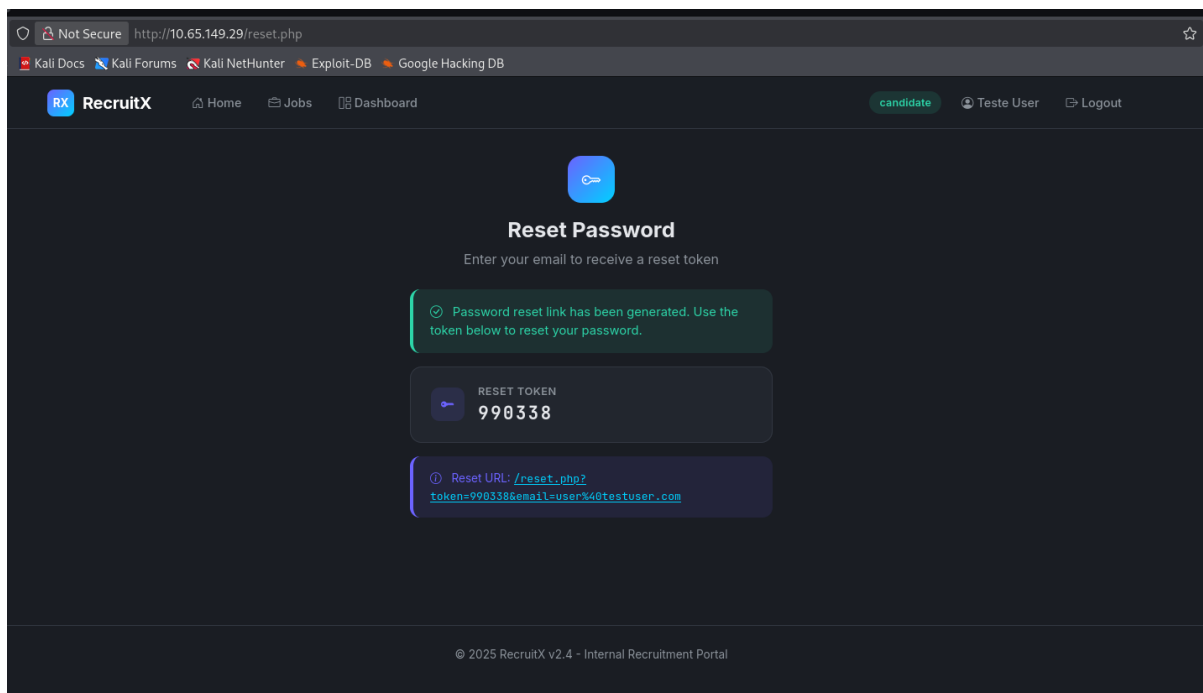
Contexto: Um token de redefinição de senha é utilizado para garantir que apenas o proprietário da conta consiga comprovar sua identidade durante o processo de recuperação de senha. No cenário atual esse token é divulgado na página de reset de senha, como consequência, terceiros que sabem e-mail de um usuário válido podem emitir um token e realizar a redefinição de senha no lugar do usuário legítimo.

Detalhes Técnicos e Evidências:

O reset de senha é feito através do seguinte link:

<http://10.65.149.29/reset.php>

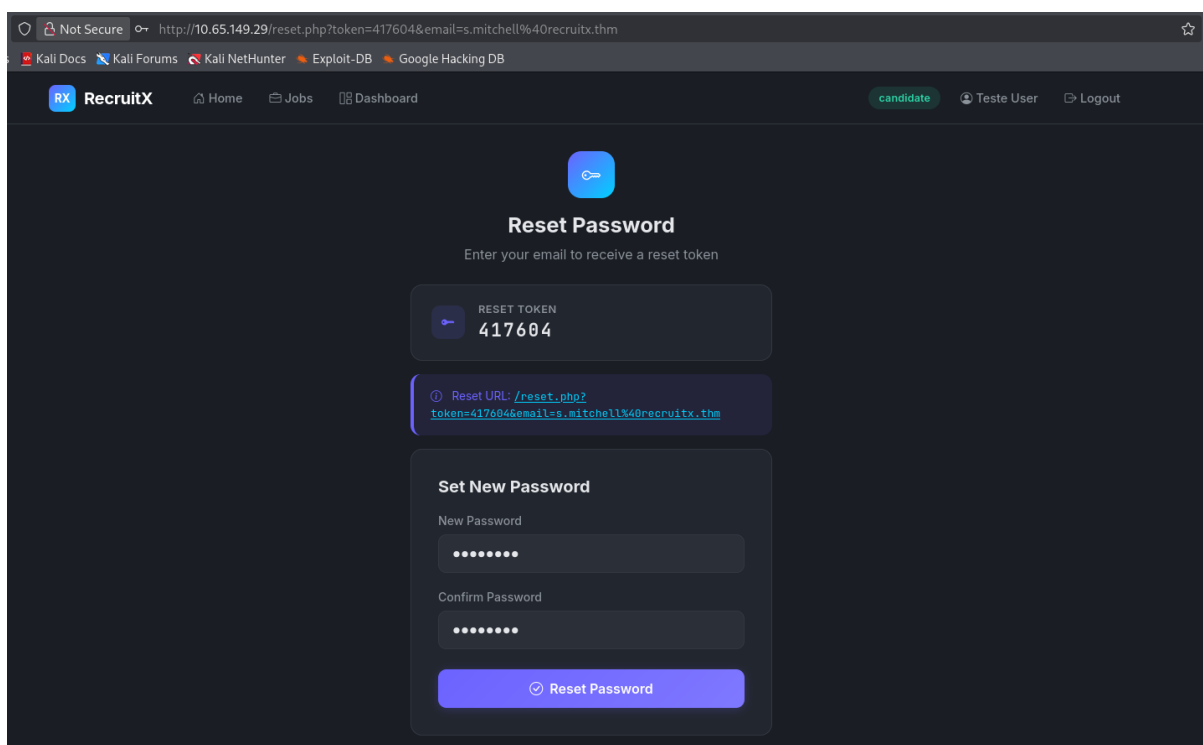
Ao acessar, digitar um e-mail de usuário válido temos o seguinte resultado:



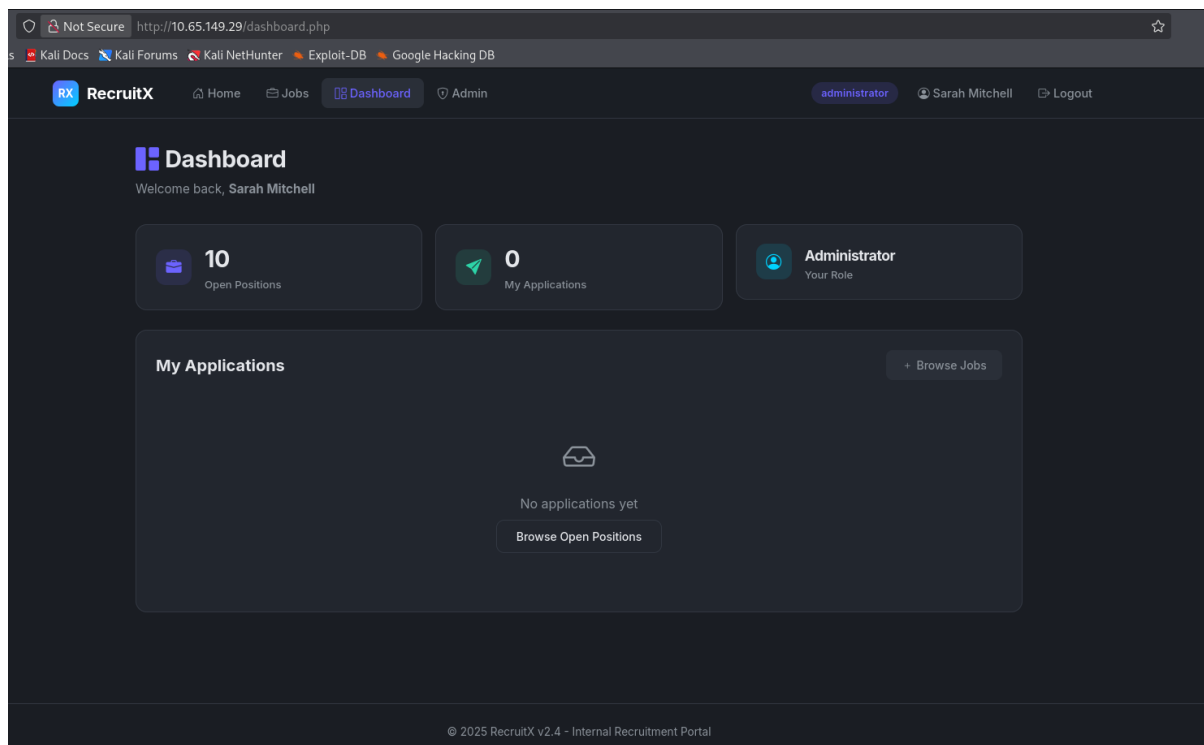
Observa-se que a aplicação retorna o token de recuperação diretamente na resposta HTTP, permitindo que qualquer usuário em posse de um endereço de e-mail válido utilize esse token para redefinir a senha da conta correspondente.

Com a vulnerabilidade de IDOR e a enumeração de usuários feitas anteriormente, foi utilizado o endereço de e-mail do usuário administrador obtido durante a exploração da vulnerabilidade de IDOR (s.mitchell@recruitx.thm).

Ao realizar o pedido de reset com o e-mail da administradora temos o seguinte retorno:



Com a alteração de senha e tentativa de acesso com o e-mail e a nova senha adicionada, temos acesso administrador do sistema:



Impacto: A exploração bem-sucedida da vulnerabilidade permite o comprometimento completo de contas de usuários legítimos, incluindo administradores da plataforma. Esse acesso pode ser utilizado para alteração de ambiente e gerenciamento indevido da plataforma. Com o acesso administrador é possível também ver as informações dos usuários e subir vagas que não existem, tirar candidatos de vagas e/ou possibilitar que candidatos não qualificados entre nas vagas.

Recomendações de Remediação:

- O token de recuperação nunca deve ser retornado na resposta da aplicação.
- O token deve ser enviado exclusivamente ao endereço de e-mail cadastrado.
- O token deve possuir alta entropia, ser imprevisível e ter uso único.
- Deve possuir tempo curto de expiração (15–30 minutos, por exemplo).
- Após a utilização, o token deve ser invalidado imediatamente.
- A aplicação deve retornar mensagens genéricas durante a recuperação de senha.

Exemplo:

Caso exista uma conta associada ao endereço informado, uma mensagem de recuperação será enviada.

Assim evita enumeração de usuários.

Lista de bloqueio de extensões de arquivo incompleta

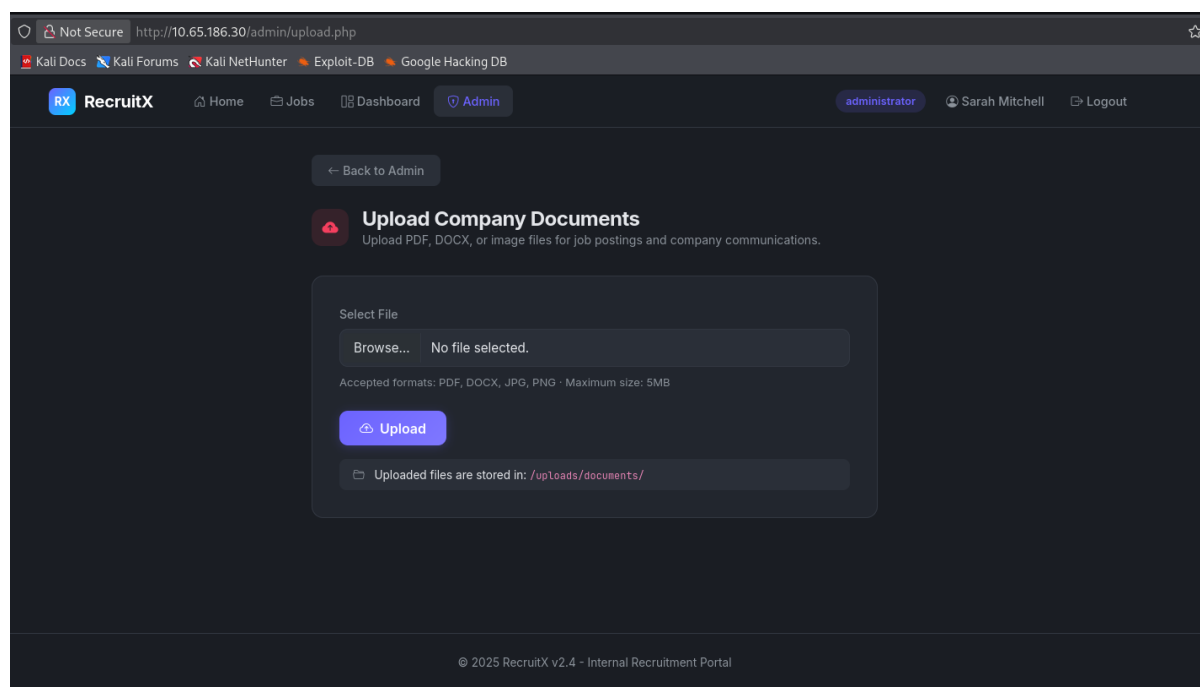
Classificação de Risco: Crítico

Resumo: Foi identificada uma falha no mecanismo de upload de documentos no dashboard de admin que permite realizar o upload de um arquivo malicioso e ter acesso ao servidor da aplicação podendo alterar dados, deletar dados, adicionar funções maliciosas para o usuário final ou execução remota de código (RCE) no servidor de hospedagem, possibilitando comprometimento da aplicação, acesso a arquivos sensíveis e execução de comandos arbitrários.

Contexto: A funcionalidade de upload deve restringir rigorosamente os tipos de arquivos aceitos, impedindo que arquivos executáveis sejam armazenados em diretórios acessíveis pelo servidor web. Quando essas validações não são implementadas corretamente, um atacante pode enviar arquivos contendo código executável, resultando em execução remota de código no servidor.

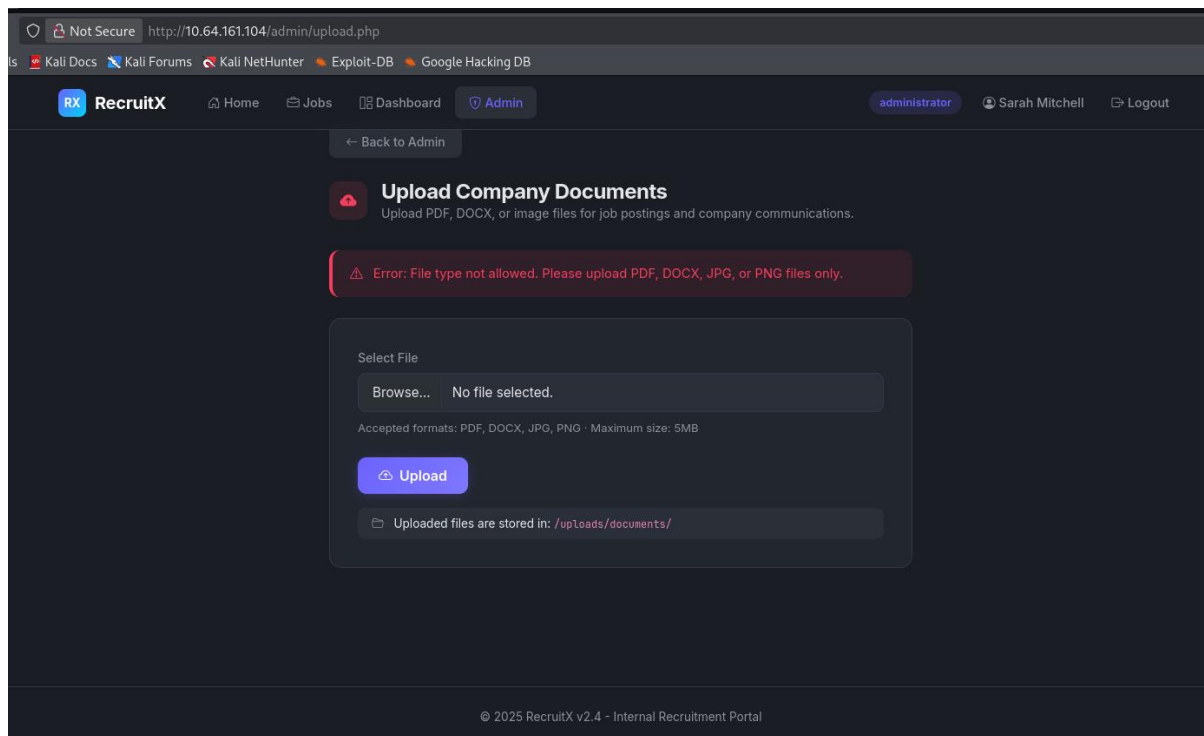
Detalhes técnicos e evidências:

Ao acessar a área de administrador que conseguimos acessar na vulnerabilidade anterior, temos uma função de upload de arquivos do administrador na seguinte tela:

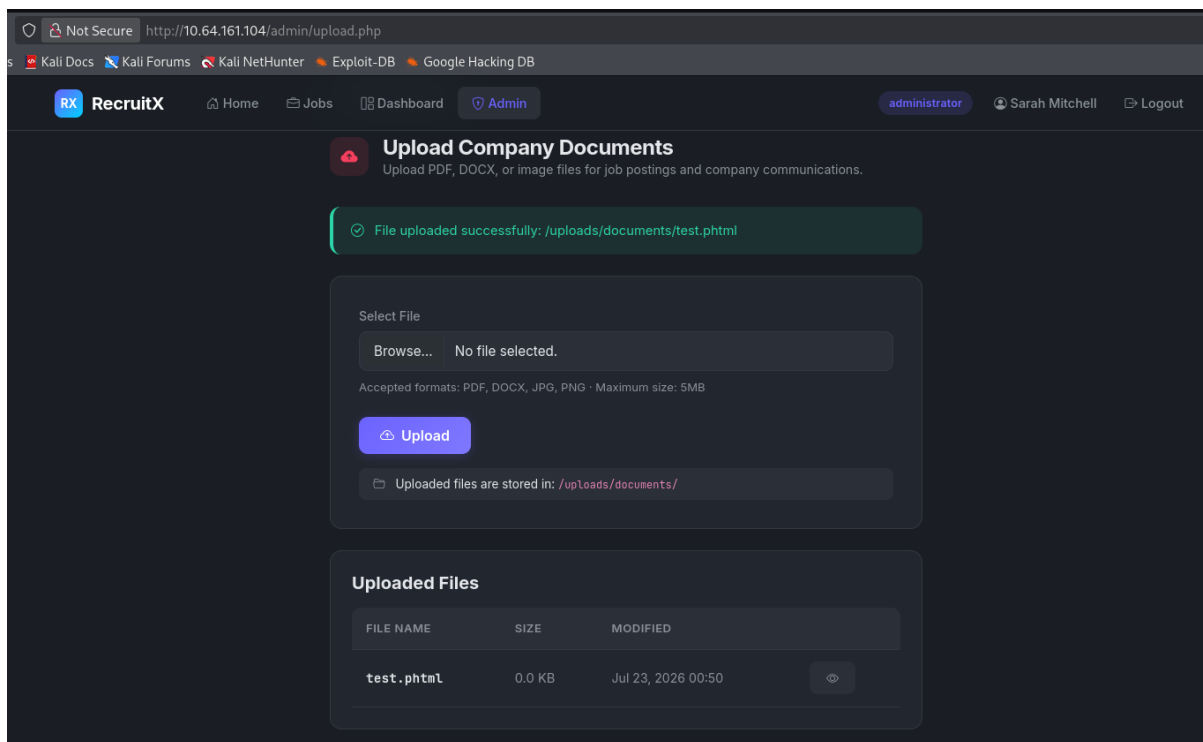


Há uma restrição do lado do cliente para aceitar o upload apenas de arquivos .pdf, .docx, .jpg e .png. Porém como isso é tratado do lado do cliente é possível subir arquivos de outros formatos.

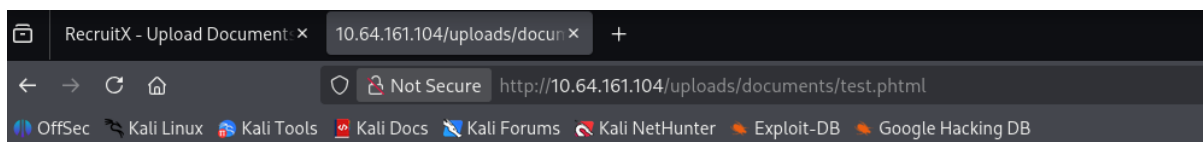
Ao tentar realizar o upload de um arquivo .txt para validar se há uma tratativa do lado do servidor obtemos um erro indicando que tem tratativa ao lado do servidor:



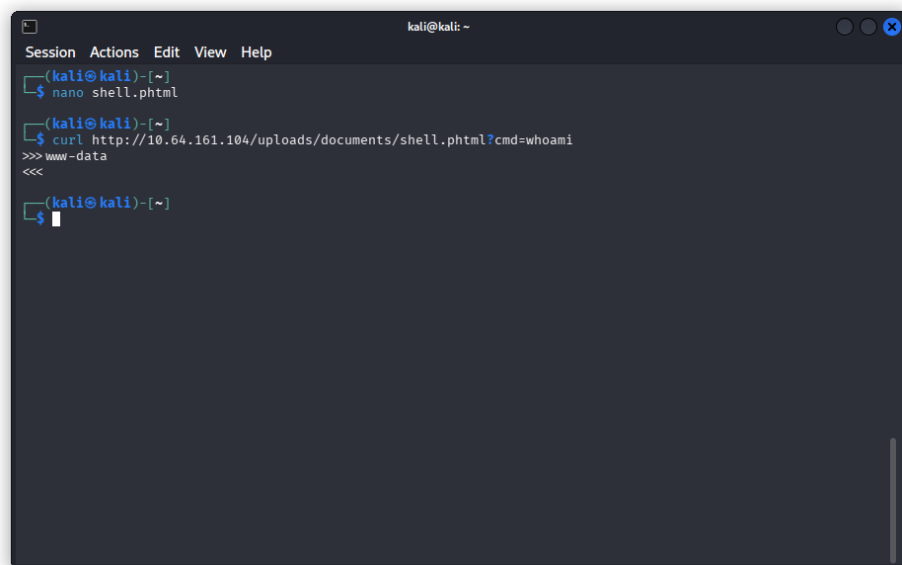
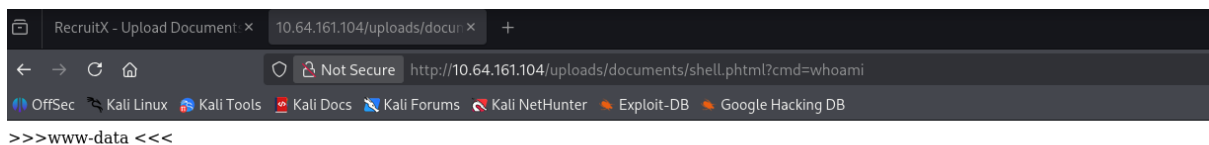
Com o reconhecimento feito no início do pentest foi identificado que a aplicação roda em um servidor Apache/2.4.58 (Ubuntu), diante disso tentamos subir um arquivo com extensão .phtml, onde o servidor Apache pode estar configurado para interpretar arquivos com extensão .phtml como scripts PHP. Dessa forma o upload foi aceito pelo servidor sem bloqueios adicionais, demonstrando que a validação realizada não restringe adequadamente extensões interpretadas pelo servidor web:



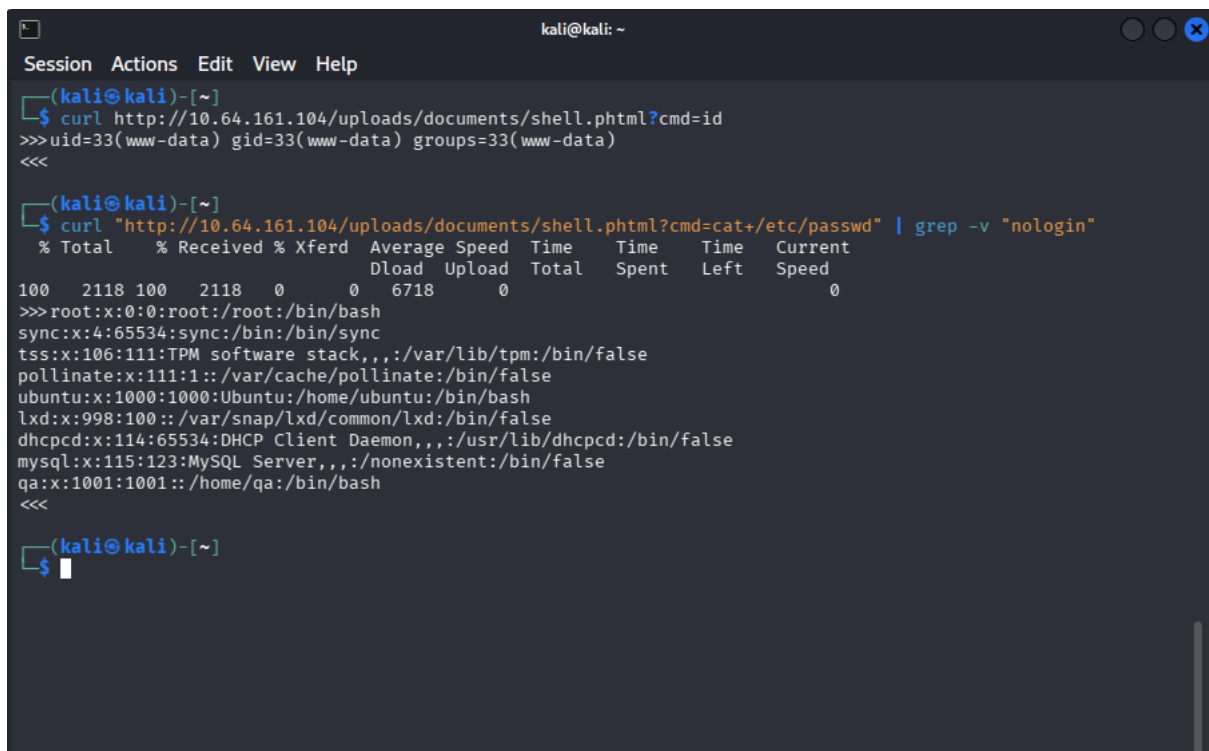
Ao acessar /uploads/documents/test.phtml temos a execução do código PHP do arquivo. Isso confirma que o arquivo enviado não foi tratado apenas como conteúdo estático, mas interpretado pelo servidor como código PHP executável:



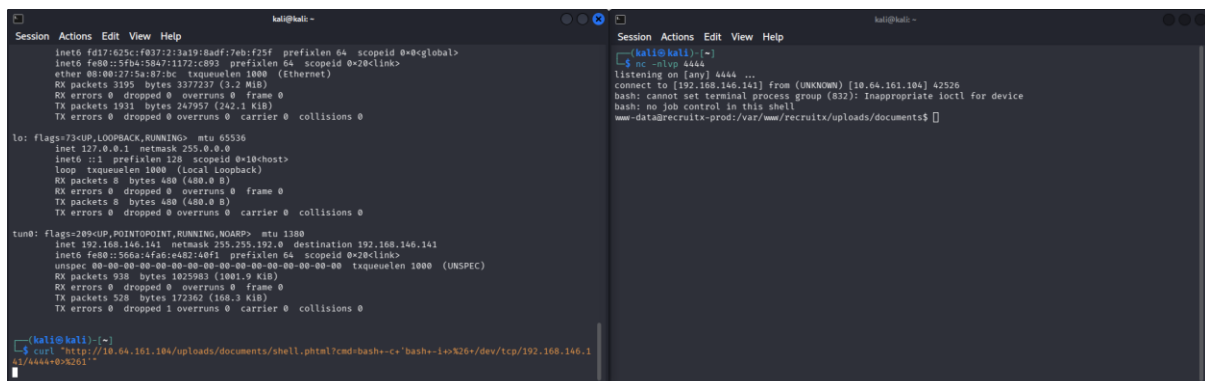
Diante dessa possibilidade de gerar um código PHP malicioso que retorna o shell do servidor fazendo com que possamos acessar os dados do servidor:



Como prova da execução remota de comandos, foi realizada a leitura do arquivo /etc/passwd, tradicionalmente utilizado em ambientes Linux para armazenar informações sobre contas do sistema:

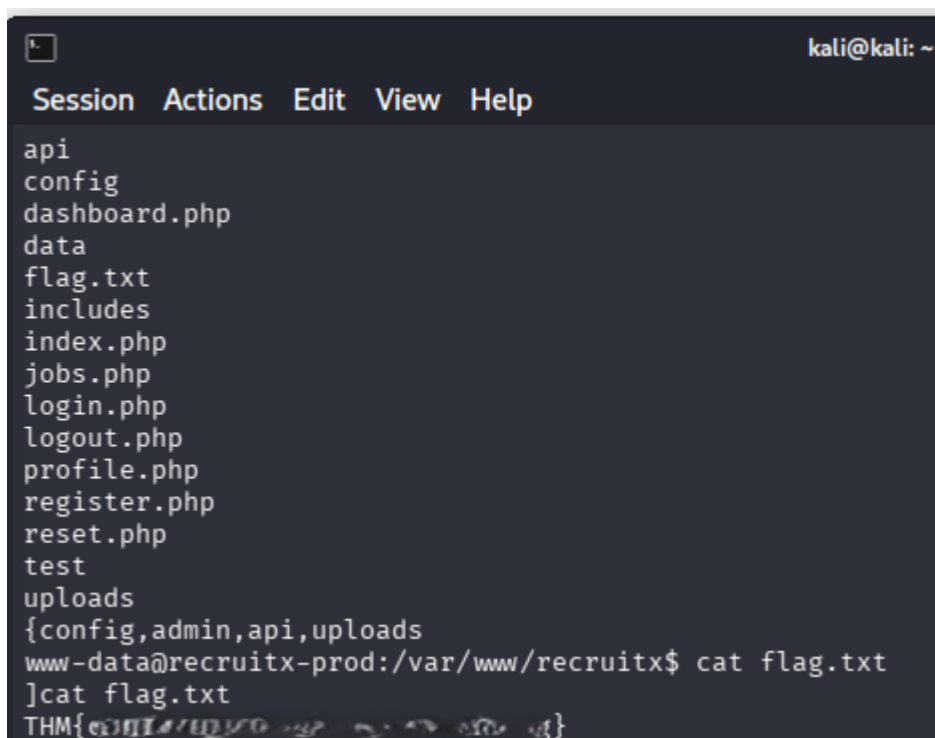


Foi estabelecida uma conexão reversa entre o servidor comprometido e a máquina, confirmando a possibilidade de execução arbitrária de comandos e controle remoto do ambiente:



The image shows two terminal windows. The left window displays network interface statistics for 'lo' and 'tun0'. The right window shows a netcat listener on port 4444. A connection is established from 192.168.146.141. The user 'www-data@recruitx-prod' is prompted for a password and enters 'var/www/recruitx/uploads/documents'. A reverse shell is then executed, showing a prompt for 'kali@kali'.

Isso possibilitou capturarmos a flag da máquina alvo:



The image shows a terminal window with a file listing of a web application. The files listed are: api, config, dashboard.php, data, flag.txt, includes, index.php, jobs.php, login.php, logout.php, profile.php, register.php, reset.php, test, uploads, {config,admin,api,uploads}, www-data@recruitx-prod:/var/www/recruitx\$, cat flag.txt,]cat flag.txt, and THM{...}. The flag.txt file is successfully accessed and its contents are displayed.

Impacto: A exploração bem-sucedida desta vulnerabilidade permitiu a execução remota de código no servidor que hospeda a aplicação. Como consequência, um atacante pode acessar arquivos sensíveis, modificar o código-fonte da aplicação, instalar mecanismos de persistência, utilizar o servidor para distribuição de malware, comprometer a disponibilidade do serviço e realizar movimentação lateral para outros ativos da infraestrutura, caso existam.

Recomendações de Remediação:

Use uma lista de permissões em vez de uma lista de bloqueios. Permita apenas extensões específicas e esperadas. Valide o conteúdo do arquivo (MIME type) além da extensão. Armazene os arquivos enviados fora da raiz da web.

É aconselhável ter uma lista de permissões para permitir arquivos válidos no upload do servidor, garantindo que somente extensões específicas e esperadas sejam aceitas. Além disso é necessário validar o conteúdo do arquivo (MIME type) além da extensão. Armazene os arquivos enviados fora da raiz da web.

Outras recomendações seria:

- Renomear arquivos enviados.
- Remover permissões de execução do diretório de upload.
- Armazenar uploads em um serviço de armazenamento (S3, Blob Storage etc.) ou fora da raiz pública.
- Bloquear interpretação de PHP dentro da pasta uploads. (O diretório de upload não deve permitir a execução de scripts.)

Cadeia de exploração (Attack chain)

Reconhecimento (Nmap)

